WEB APPLICATION CYBERSECURITY ASSESSMENT

Complete Ethical Hacking & Student Training Guide

Assessment reference:
https://www.ictacademy.in/CSR/dashboard/Trainer_Dashboard.aspx

Reconnaissance • Authentication • Authorization • Injection • APIs
Infrastructure • DevSecOps • Monitoring • Reporting

Purpose

A practical framework for cybersecurity engineers and students to assess a web application methodically. It explains what to test, why it matters, where to look, how to test safely, which tools to use, what outputs to record, and the limitations of each approach. It is designed as a training and reporting guide—not as a claim that the target website contains vulnerabilities.

IMPORTANT — AUTHORIZATION

The live ICT Academy site must only be tested within explicit written authorization and agreed scope. A public login page is not permission to bypass authentication, enumerate accounts, access another trainer's data, modify records, fuzz endpoints, exploit vulnerabilities, or run load tests. Use passive observations for initial reconnaissance and use a local lab or staging environment for active testing unless the owner expressly authorizes otherwise.

Prepared as a classroom reference • October 2026

1. Assessment Scope and Methodology

The supplied URL ends in .aspx, which is consistent with an ASP.NET-style application, but this alone does not prove the actual framework version, programming language version, database, hosting provider, or security posture. Confirm technology using multiple observable signals and mark confidence in the report.

Assessment stages

- **1. Define scope:** written permission, exact domains/IPs, allowed test types, test window, rate limits, data-handling rules and emergency contact.
- **2. Reconnaissance:** public DNS, TLS, headers, technology clues, published pages and architecture information.
- **3. Application tests:** authentication, session management, authorization, input validation, APIs and business logic.
- **4. Platform tests:** infrastructure, dependencies, secrets, logging and cloud configuration when separately authorized.
- **5. Validate:** verify findings safely, distinguish evidence from assumptions and assess impact.
- **6. Report and retest:** remediation guidance, owner, priority, retest result and closure status.

Useful standards and references

- OWASP Web Security Testing Guide (WSTG) — web testing methodology.
- OWASP Top 10 — common web application risk categories.
- OWASP ASVS — verifiable application security requirements.
- OWASP API Security Project — API-specific risk areas.
- OWASP Cheat Sheet Series — implementation and remediation guidance.

Evidence labels

Label	Meaning
Verified fact	Directly supported by collected evidence.
Suspected issue	A signal suggests a possible weakness, but validation is incomplete.
Not tested	No assessment was performed or scope/access was insufficient.
Not applicable	The feature or control does not apply, with a short rationale.
Confirmed vulnerability	A weakness and its security impact were safely validated.

2. Reconnaissance and Technology Discovery

Test	What / why	Where	Tools	Output
1. Domain discovery	Identify known public domains and in-scope hostnames.	Public DNS and official links.	nslookup, dig	Domain inventory; record source and confidence.
2. DNS security	Review published DNS records and email protections.	TXT, MX, CAA and other published records.	dig, nslookup, MXToolbox	DNS observations; do not equate absence with exploitability.
3. Subdomain inventory	Identify known public subdomains without intrusive probing.	Public sources and passive certificate data.	Certificate Transparency, Amass passive mode	Candidate subdomains; validate scope before testing.
4. Technology fingerprinting	Identify likely frameworks, CMSs and libraries.	Response headers, HTML, JS and visible behavior.	Wappalyzer, WhatWeb	Technology list with confidence and supporting evidence.
5. Server identification	Record server information disclosed in responses.	HTTP response headers.	DevTools, curl	Header observations; headers can be absent or misleading.
6. TLS certificate review	Check certificate validity and hostname coverage.	HTTPS certificate chain.	Browser, SSL Labs	Certificate and hostname results.
7. TLS configuration	Review protocol and cipher configuration.	Approved HTTPS endpoint.	SSL Labs, testssl.sh	TLS report; active checks only within scope.
8. Security headers	Review browser-side protections.	HTTP responses.	DevTools, ZAP passive scan	Header checklist; missing headers need context.
9. Public files	Identify unintentionally published documents or configuration artifacts.	Publicly linked or permitted resources.	Browser, passive crawler	Exposure inventory; do not access sensitive files beyond scope.
10. Source-map exposure	Check if source maps or debug artifacts expose implementation details.	Resources already served to the browser.	DevTools, source-map review	Exposure evidence; source maps are not automatically a vulnerability.
11. Public indexing	Identify indexed pages and public documents.	Search engine results.	Search engines	Public exposure inventory.
12. Architecture mapping	Understand visible entry points and trust boundaries.	Approved documentation and observable redirects.	Browser, diagramming tools	High-level architecture map.

Pros: low impact, builds an attack-surface baseline and helps prioritize further review. **Cons:** fingerprints may be inaccurate; an observation alone does not prove a vulnerability. **Deliverable:** asset inventory with evidence, confidence, risk and follow-up.

3. Authentication and Identity Security

Test	What / why	Where	Tools	Output
13. Login validation	Verify invalid credentials are handled safely.	Login page with approved test accounts.	Browser, Burp Suite	Authentication results.
14. Password policy	Review minimum length and password handling rules.	Account settings and documented policy.	Browser, policy review	Password policy assessment.
15. MFA	Determine whether MFA is available or required.	Login and account settings.	Browser, configuration review	MFA coverage.
16. Password recovery	Review token expiry, single-use behavior and account-change safeguards.	Recovery workflow.	Browser, Burp Suite in scope	Reset workflow evidence.
17. Account enumeration	Check whether responses reveal account existence.	Login and recovery messages.	Manual testing, Burp Suite	Message consistency results.
18. Brute-force protection	Verify reasonable throttling and account protection.	Dedicated test accounts and agreed limits.	Burp Suite, approved harness	Rate-limit observations; avoid account disruption.
19. Session creation	Review session establishment after login.	Login response and browser cookies.	DevTools, Burp Suite	Session lifecycle evidence.
20. Session expiration	Check idle and absolute expiry against policy.	Inactive test session.	Browser, approved procedure	Expiration results.
21. Logout behavior	Verify logout invalidates the session server-side.	Logout followed by an authorized session check.	Browser, proxy	Logout verification.
22. Password change	Check whether old sessions are handled correctly.	Account security settings.	Browser, test account	Session invalidation results.
23. Cookie security	Review Secure, HttpOnly and SameSite attributes.	Response headers and browser storage.	DevTools, ZAP	Cookie configuration report.
24. Authentication bypass resistance	Verify protected routes reject unauthenticated access.	Known authorized routes.	Burp Suite, ZAP	Access-control evidence.

Pros: helps detect account takeover and session weaknesses. **Cons:** lockout tests can disrupt accounts and trigger monitoring; use dedicated test accounts and agreed limits. **Deliverable:** authentication coverage matrix and session lifecycle diagram.

4. Authorization and Privilege Escalation

Test	What / why	Where	Tools	Output
25. Horizontal access control	One trainer must not access another trainer's records.	Two synthetic test users and records.	Burp Suite, browser	Cross-user access matrix.
26. Vertical access control	Trainer privileges must not grant admin powers.	Separate test roles.	Browser, proxy	Role enforcement results.
27. Direct URL access	Protected pages must enforce permissions when accessed directly.	Known permitted test routes.	Browser	Direct-access results.
28. Parameter authorization	Server must verify ownership of requested records.	Approved record retrieval requests.	Burp Suite in lab/scope	IDOR/BOLA assessment.
29. Function-level authorization	Restricted actions must be blocked for unauthorized roles.	Role-specific forms and actions.	Browser, code review	Function permission matrix.
30. HTTP method authorization	Read/write permissions must be enforced per operation.	Approved endpoints.	Burp Suite	Method/permission matrix.
31. Role modification	Users must not grant themselves higher privileges.	Profile and role-management workflows.	Code review, synthetic accounts	Role integrity evidence.
32. Tenant isolation	Organizations' data must remain separated, if multi-tenant.	Synthetic tenants.	Integration tests, approved proxy	Tenant isolation results.
33. Administrative controls	Admin actions must require correct authorization.	Approved administrator test account.	Browser, code review	Admin control assessment.

Example: Trainer A should not be able to read or modify Trainer B's record merely by knowing its identifier. The server must check identity, ownership and permission on every relevant request.

Pros: directly tests confidentiality and integrity boundaries. **Cons:** requires accurate role definitions and synthetic data to avoid false positives or unauthorized exposure. **Deliverable:** role-permission matrix with Allowed, Denied, Not applicable and Not tested states.

5. Input Validation and Injection Security

Test	What / why	Where	Tools	Output
34. SQL injection	Check whether input can alter database query behavior.	Applicable search and form fields.	Code review, approved DAST	Assessment; use parameterized queries.
35. Reflected XSS	Check whether request input is safely rendered.	Query parameters and reflected fields.	Burp Suite, ZAP in lab	Output-encoding results.
36. Stored XSS	Check whether saved input executes as content.	Applicable profile/comment fields.	Browser, code review, lab proxy	Stored-content results.
37. DOM-based XSS	Review unsafe client-side DOM operations.	JavaScript processing untrusted input.	DevTools, code review	Client-side findings.
38. CSRF	Check protection for sensitive state-changing actions.	Forms and requests that change state.	DevTools, Burp Suite in scope	CSRF assessment.
39. Command injection	Review whether input reaches operating-system commands.	Only applicable server-side code paths.	Source review, isolated lab	Code-path analysis.
40. Path traversal	Check file path handling and directory boundaries.	File download/document features, if present.	Code review, lab	Path-handling results.
41. XML external entities	Review unsafe XML parsing if XML is accepted.	XML imports or XML APIs.	Dependency review, lab tests	XML parser assessment.
42. SSRF	Review user-controlled server-side outbound requests.	URL fetchers, imports or webhooks, if present.	Code review, isolated lab	SSRF risk assessment.
43. Template injection	Check if untrusted data is interpreted as template syntax.	Template-driven features, if present.	Code review, lab tests	Template safety report.
44. Validation bypass	Verify server-side validation independent of browser controls.	Forms and API fields.	Unit tests, proxy	Validation results.
45. Output encoding	Ensure data is encoded for its output context.	HTML, attributes, URLs and script contexts.	Code review, test suite	Encoding coverage.

Safety: use a dedicated lab or explicitly approved scope. Injection testing can alter records, trigger server-side actions or affect availability. Automated scanners can report false positives. Validate findings safely and document the affected code path, impact and fix.

6. Business Logic and Workflow Security

Test	What / why	Where	Tools	Output
46. Workflow sequencing	Required steps must not be skipped.	Multi-step processes.	Manual test cases, Playwright	Workflow coverage.
47. State transitions	Prevent invalid status changes.	Submission/approval workflows, if present.	API tests, unit tests	State-transition matrix.
48. Duplicate submissions	Check duplicate handling and idempotency.	Submit/update operations.	Postman, automated tests	Duplicate-operation results.
49. Data tampering	Sensitive fields must not be changed by unauthorized users.	Forms and API bodies.	Burp Suite in lab	Field-level permission matrix.
50. Approval separation	Check whether policy requires independent approval.	Approval workflows, if present.	Manual tests, code review	Separation-of-duties results.
51. Business limits	Validate quantities, dates, statuses and other rules.	Applicable fields.	Unit tests, API tests	Boundary-value results.
52. Race conditions	Check concurrent operations against business invariants.	Authorized staging tests.	Test harness, logs	Concurrency test results.
53. Auditability	Attribute important actions to the correct user.	State-changing operations.	Logs, code review	Audit event coverage.

Pros: detects workflow flaws that scanners often miss. **Cons:** needs domain knowledge; concurrency tests need a non-production environment and careful limits. **Deliverable:** business rules, negative tests and evidence for failed rules.

7. API and Integration Security

Test	What / why	Where	Tools	Output
54. API inventory	Understand endpoints and methods.	Approved docs and browser Network panel.	DevTools, Postman	API inventory.
55. API authentication	Protected endpoints require valid authentication.	API requests with test accounts.	Postman, Burp Suite	Authentication results.
56. API authorization	Verify record/function permissions server-side.	Synthetic accounts and records.	Postman, test suite	API access matrix.
57. Schema validation	Reject malformed or unexpected fields.	Request bodies and parameters.	OpenAPI validators, unit tests	Schema validation report.
58. Excessive data exposure	Return only necessary fields.	API responses.	DevTools, schema review	Data minimization report.
59. Mass assignment	Prevent unexpected fields from changing protected properties.	Update APIs, if present.	Code review, API tests	Field-binding results.
60. API rate limiting	Check resource-specific limits.	Approved API test environment.	Postman, controlled scripts	Rate-limit report.
61. API versioning	Identify obsolete versions and support status.	Documented/observed routes.	Inventory, code review	API lifecycle inventory.
62. Third-party integrations	Review permissions and data sharing.	Documented integrations.	Architecture review, secret scanning	Integration risk register.

Pros: extends coverage beyond the user interface. **Cons:** endpoint discovery and active probing must stay within scope; server specs/logs may be needed. **Deliverable:** API security assessment mapped to the OWASP API Security Project.

8. File Handling and Data Protection

Test	What / why	Where	Tools	Output
63. Upload validation	Enforce file type and size on server.	Upload features, if present.	Unit tests, approved files	Upload validation matrix.
64. File content validation	Compare declared type with actual content.	Test fixtures in staging.	File inspection, code review	Content validation results.
65. Storage isolation	Avoid executable or unintentionally public uploads.	Storage config and test files.	Code review, storage audit	Storage assessment.
66. Download authorization	Private documents require proper permission.	Document retrieval workflow.	Test accounts, browser	Download permission matrix.
67. Sensitive data exposure	Avoid unnecessary confidential data in responses.	Authorized responses and logs.	DevTools, data-flow review	Exposure register.
68. Encryption at rest	Verify database/storage encryption controls.	Configuration and cloud settings.	Cloud console, IaC review	Encryption evidence.
69. Data retention	Verify retention and deletion requirements.	Policies and data model.	Documentation, authorized SQL review	Retention assessment.
70. Backup security	Review access, encryption and restore capability.	Approved infrastructure review.	Backup tools, recovery drills	Backup control report.

Pros: protects records and uploaded documents. **Cons:** deeper validation often requires configuration access and organizational policies. **Deliverable:** data-flow diagram, sensitive-data inventory, access evidence and remediation plan.

9. Server, Hosting and Infrastructure Security

Test	What / why	Where	Tools	Output
71. Server hardening	Identify unnecessary services and insecure defaults.	Authorized server configuration.	CIS Benchmarks, configuration audit	Hardening checklist.
72. Patch management	Identify outdated/unsupported software.	Approved software inventory.	Vendor advisories, vulnerability scanners	Patch risk register.
73. Database security	Review permissions, network exposure and encryption.	Database configuration.	Native database tools, config review	Database security report.
74. Least privilege	Limit service identities to required permissions.	Service accounts and IAM policies.	IAM review, policy analyzers	Privilege matrix.
75. Network segmentation	Isolate application, database and management tiers.	Network diagrams and firewall rules.	Cloud console, firewall review	Network diagram.
76. WAF configuration	Review rules, logging and exceptions, if deployed.	WAF settings.	WAF console, config review	WAF control report.
77. Secrets management	Check credentials, tokens and keys handling.	Authorized repositories/config.	Gitleaks, secret scanners	Secret exposure report.
78. Cloud security	Review public exposure, identity and storage settings.	Authorized cloud account.	Prowler, ScoutSuite, native tools	Cloud findings.
79. Backup recovery	Verify restore process and recovery objectives.	Staging recovery exercise.	Backup tools, runbooks	Recovery test report.
80. Availability protections	Review resilience and resource limits.	Architecture and configuration.	Metrics, config audit	Availability risk assessment.

Scope: these tests require infrastructure access or explicit authorization beyond ordinary public browsing. **Pros:** finds risks not visible from the frontend. **Cons:** needs coordination, credentials and configuration access. **Deliverable:** infrastructure findings, architecture, configuration evidence and hardening actions.

10. Dependency, Source Code and CI/CD Security

Test	What / why	Where	Tools	Output
81. SAST	Find insecure code patterns.	Authorized source repository.	Semgrep, SonarQube	Static analysis report.
82. SCA	Identify vulnerable dependencies.	Manifests and lockfiles.	OWASP Dependency-Check, OSV-Scanner	Dependency findings.
83. Secret scanning	Detect committed credentials.	Authorized repository and history.	Gitleaks, GitHub secret scanning	Secret findings; handle secrets carefully.
84. IaC review	Find insecure infrastructure configuration.	Terraform and deployment manifests.	Checkov	Policy violations.
85. Container security	Identify vulnerable packages and settings.	Container images, if used.	Trivy	Image scan report.
86. CI/CD access control	Review who can change workflows/releases.	Build platform settings.	Platform settings and audit logs	Pipeline permissions matrix.
87. Artifact integrity	Check build artifacts are traceable and protected.	Build/release process.	Signing and provenance tools	Build integrity evidence.
88. Dependency updates	Track and verify dependency fixes.	Update workflow.	Renovate, Dependabot	Remediation status.
89. Regression tests	Prevent previously fixed defects from returning.	Test suites and CI.	pytest, Playwright, pipeline tests	Automated results.

Pros: enables repeatable early detection and cheaper fixes. **Cons:** scanners produce false positives and cannot prove the entire app is secure. Coverage depends on what was scanned. **Deliverable:** reproducible CI security pipeline, findings and remediation tracking.

Tool references: Semgrep · SonarQube · Trivy · Checkov · OWASP Dependency-Check.

11. Logging, Monitoring and Incident Response

Test	What / why	Where	Tools	Output
90. Authentication logging	Record relevant login successes/failures.	Application and identity logs.	SIEM, log review	Event coverage.
91. Privileged activity	Track administrative and sensitive changes.	Audit logs.	SIEM, native audit tools	Privileged activity report.
92. Log integrity	Restrict access and protect logs from changes.	Logging platform.	Platform audit tools	Log protection evidence.
93. Secret redaction	Avoid logging passwords, tokens and sensitive data.	Logs and traces.	Log queries, code review	Redaction results.
94. Alert coverage	Verify high-risk events generate appropriate alerts.	Detection rules and test events.	Sentinel, Splunk or equivalent	Alert test matrix.
95. Incident response	Evaluate response to simulated compromise.	Tabletop exercise/runbooks.	Incident tracking tools	Response report.
96. Remediation lifecycle	Track findings through closure.	Ticketing and remediation workflow.	Jira, ServiceNow	Vulnerability dashboard.

Pros: improves detection, accountability and investigations. **Cons:** excessive logs create noise and privacy risks; detections need tuning. **Deliverable:** logging matrix, alert tests, incident runbook and remediation dashboard.

12. Consolidated Tool Selection Guide

Tool	Primary purpose	Typical output	Main limitation
Browser DevTools	Inspect requests, responses, cookies and resources.	Network evidence and browser findings.	Cannot reveal all server-side behavior.
Wappalyzer	Technology fingerprinting.	Suspected technology list.	Can be inaccurate.
curl	Inspect HTTP behavior.	Headers and response body.	Not a vulnerability scanner.
DNS utilities	Inspect DNS records.	DNS observations.	Do not prove app vulnerabilities.
SSL Labs	Public TLS assessment.	TLS results.	Limited to supported checks.
Burp Suite	HTTP interception and app testing.	Request/response evidence.	Active tests need authorization.
OWASP ZAP	Web analysis and scanning.	Alerts and reports.	False positives/incomplete coverage.
Postman	API testing.	API responses and test results.	Does not automatically prove authorization.
Playwright	Browser automation and regression tests.	Results and traces.	Needs sound test design.
Semgrep	Source code security analysis.	Code findings.	Depends on rules and code coverage.
Gitleaks	Secret detection.	Potential secret findings.	Validate results and handle secrets securely.
Trivy	Dependency, container and config scanning.	Vulnerability reports.	Depends on artifacts and databases.
Checkov	Infrastructure-as-code review.	Policy violations.	Does not assess every runtime condition.
Prowler	Cloud security posture assessment.	Cloud configuration findings.	Requires appropriate cloud access.
Wireshark	Authorized network traffic analysis.	Packet captures.	HTTPS payloads are not automatically readable.
Sentinel / Splunk	Security monitoring and investigation.	Alerts and incident records.	Requires telemetry and detection engineering.
Jira / ServiceNow	Finding/remediation tracking.	Tickets and audit trail.	Does not discover vulnerabilities.

Official tool references

- Burp Suite documentation

- OWASP ZAP documentation
- Playwright
- Prowler

13. Required Student Deliverables

Deliverable	Required contents
1. Asset and architecture inventory	Domains, confirmed technologies, endpoints, data flows and trust boundaries.
2. Security test matrix	Test ID, objective, scope, expected behavior, actual result and evidence.
3. Validated vulnerability report	Finding, severity, impact, reproduction steps, root cause and remediation.
4. Remediation plan	Owner, priority, target date, recommended fix and retest status.
5. Student lab portfolio	Synthetic vulnerable app, scripts, screenshots, fixed code and regression tests.

14. Severity and Prioritization

Priority	Illustrative scenario	Suggested response
Critical	Verified unauthenticated compromise of highly sensitive systems.	Immediate containment and escalation.
High	Verified cross-user access to sensitive records.	Urgent remediation and retesting.
Medium	Weakness with limited exploitability or impact.	Schedule a risk-based fix.
Low	Minor hardening issue without demonstrated significant impact.	Address through planned improvements.
Informational	Observation or improvement recommendation.	Document and assess.

These are illustrative scenarios, not findings about ICT Academy. A real rating requires evidence, affected assets, business impact and appropriate scoring criteria. CVSS can support technical severity scoring; business context should guide remediation priority.

15. Suggested Four-Week Training Roadmap

Week	Focus	Student activities
1 — Foundation	Reconnaissance and web fundamentals	HTTP, DNS, TLS, DevTools, fingerprinting, OWASP methodology and scope.
2 — Identity	Authentication, sessions and authorization	Cookie handling, role-based access, IDOR/BOLA, privilege boundaries.
3 — Application	Injection, APIs and business logic	Use an intentionally vulnerable lab, validate inputs and test fixes.
4 — Engineering	Infrastructure, DevSecOps and reporting	Dependency scanning, cloud posture, logs, risk scoring, remediation and presentation.

A four-week course provides a foundation; it does not make students experts in every domain. Adjust duration to student experience and lab availability.

16. Master Assessment Checklist

Reconnaissance

- ■ Scope and written authorization
- ■ DNS, TLS and technology inventory
- ■ Public exposure and security headers

Identity and access

- ■ Login and password recovery
- ■ Session and cookie security
- ■ Role-based access and cross-user isolation

Application

- ■ Input validation and output encoding
- ■ Injection and CSRF review

■ Business logic and workflow rules

Data and APIs

■ API authentication and authorization

■ File handling and data exposure

■ Encryption, retention and backup controls

Platform and operations

■ Server and cloud configuration

■ Dependency and secrets scanning

■ Logging, alerting and incident response

Reporting

■ Validate evidence and severity

■ Document fixes and responsible owners

■ Retest and close findings

17. Safe Local Lab: Authorization Testing

Objective: demonstrate broken access control without accessing real users' private data.

- Create two synthetic test users, Trainer A and Trainer B, with separate dummy records.
- Log in as Trainer A and retrieve Trainer A's own record.
- Using an instructor-approved test request, attempt to access Trainer B's synthetic record.
- Verify the server denies access even if a record identifier is changed.
- Repeat with an unauthenticated request and a separate administrator test role.
- Record expected versus actual behavior, evidence, impact and remediation.

Expected secure result: Trainer A cannot read or modify Trainer B's record. The backend verifies identity and authorization on every relevant request.

18. Sample Finding Report Template

Field	What to record
Finding ID / title	Unique ID and concise name.
Affected asset	Approved hostname, route, component or code location.
Scope / authorization	Engagement reference and permitted test scope.
Evidence	Redacted request/response, screenshot, log or code reference.
Expected vs actual	What should happen compared with what was observed.
Impact	Confidentiality, integrity, availability and business impact.
Severity / confidence	Justified rating, evidence strength and assumptions.
Root cause	Underlying control or coding issue, where known.
Remediation	Specific defensive fix and relevant references.
Retest / status	Retest result, date, owner and closure state.

19. Final Recommendations

- Start with reconnaissance, authentication, authorization and business logic to establish what the application exposes and whether permissions match intended design.
- Expand into server, cloud and CI/CD assessment only when relevant access and authorization are in place.

- Do not label a scanner alert as Critical or High without validating evidence and impact.
- Keep credentials, tokens, personal information and sensitive request/response data out of shared reports; redact evidence.
- Clearly separate verified facts, suspected issues, untested areas, not-applicable controls and confirmed vulnerabilities.

Closing note

A credible cybersecurity assessment is not a list of tools or attempted exploits. It is a documented, authorized and reproducible process that distinguishes facts from assumptions, explains business risk, recommends fixes and verifies that those fixes work.